Introduction:-

Keras is an open-source high-level Deep Learning library that provides a simple and user-friendly interface for building, training, and deploying neural networks. It is designed to make Deep Learning easy to learn while remaining powerful enough for real-world applications.

Keras is tightly integrated with **TensorFlow** and runs on top of it as its official high-level API. It allows developers to create complex Deep Learning models using just a few lines of code.

Keras provides pre-built components such as layers, activation functions, optimizers, and loss functions, enabling rapid model development without dealing with low-level implementation details.

History:-

Keras was created by **François Chollet**, a Google engineer, with the goal of making Deep Learning simple, fast, and accessible to everyone. Before Keras, building neural networks often required writing large amounts of complex code.

Timeline

- **2015:** Keras was created by François Chollet.
- **2015:** First open-source release of Keras.
- **2017:** TensorFlow adopted Keras as its official high-level API.
- **2019:** TensorFlow 2.0 made `tf.keras` the default API for Deep Learning.
- **Present:** Keras is one of the most widely used Deep Learning libraries.

Why Keras was Developed:-

Keras was developed to make **Deep Learning simple, fast, and user-friendly**. Before Keras, building neural networks required writing large amounts of complex code, making it difficult for beginners and time-consuming for developers.

Keras provides a high-level API that allows users to create, train, evaluate, and deploy Deep Learning models with just a few lines of code. It hides low-level implementation details, enabling developers to focus on designing and improving models.

It also offers built-in layers, activation functions, optimizers, loss functions, and other utilities that simplify the development of neural networks.

Objectives of Keras

- Simplify Deep Learning development.
- Reduce the amount of code required.
- Provide an easy-to-use API.
- Enable rapid prototyping of models.
- Improve code readability and maintainability.
- Support seamless integration with TensorFlow.

Features:-

Keras offers a wide range of features that make Deep Learning development simple, efficient, and powerful. Its high-level API allows developers to build neural networks with minimal code while providing flexibility for creating both simple and advanced models.

Major Features of Keras

1. User-Friendly API

Keras provides a simple, clean, and intuitive interface, making it easy for beginners to learn and build Deep Learning models.

2. Built on TensorFlow

Keras is the official high-level API of TensorFlow, giving users access to TensorFlow's powerful backend while keeping the code simple.

3. Modular Design

Keras is modular, allowing users to easily combine layers, activation functions, optimizers, and loss functions to create custom neural networks.

4. Fast Model Development

With pre-built components and fewer lines of code, Keras enables rapid prototyping and faster development of Deep Learning models.

5. Supports Multiple Layers

Keras includes various built-in layers such as Dense, Dropout, Flatten, Conv2D, LSTM, and many more for building different types of neural networks.

Applications:-

1. Image Classification

Keras is used to classify images into different categories, such as identifying animals, vehicles, or handwritten digits.

2. Object Detection

It helps detect and locate multiple objects within images and videos for applications like surveillance and autonomous vehicles.

3. Natural Language Processing (NLP)

Keras is used for text classification, language translation, sentiment analysis, chatbots, and text generation.

4. Speech Recognition

It enables computers to recognize and convert spoken language into text for voice assistants and transcription systems.

5. Recommendation Systems

Keras is used to build recommendation engines that suggest movies, songs, products, or videos based on user preferences.

Installation & Import:-

Before using Keras, you need to install TensorFlow because Keras is included as the official high-level API of TensorFlow.

Once installed, you can import Keras using the `tensorflow.keras` module and start building Deep Learning models.

Requirements

- Python 3.x
- TensorFlow
- Google Colab, Jupyter Notebook, or any Python IDE

```
!pip install tensorflow
```

```
from tensorflow import keras
```

```
print(keras.__version__)
```

```
3.13.2
```

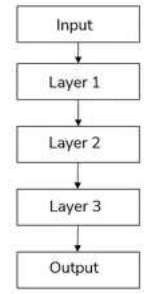
Keras Models:-

Keras provides different ways to create Deep Learning models. A model defines how layers are connected and how data flows through the neural network.

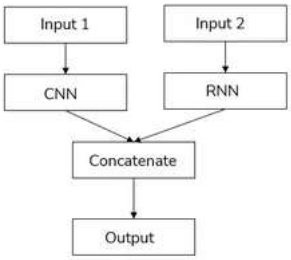
The two most commonly used model types are:

- **Sequential Model** – Layers are stacked one after another.
- **Functional API** – Used to build more complex models with multiple inputs, outputs, or branching.

Choosing the appropriate model depends on the complexity of your Deep Learning application.



Sequential Model



Functional Model

1. Sequential():-

The `Sequential()` model is the simplest way to build a neural network in Keras. It creates a linear stack of layers where each layer has exactly one input and one output.

It is best suited for beginners and simple Deep Learning models.

```
from tensorflow import keras

model = keras.Sequential()

print(model)
```

```
<Sequential name=sequential, built=False>
```

2. Functional API:-

The Functional API is used to build more flexible and complex neural networks. It supports models with multiple inputs, multiple outputs, shared layers, and non-linear architectures.

It is commonly used for advanced Deep Learning applications.

```
from tensorflow import keras

inputs = keras.Input(shape=(4,))
outputs = keras.layers.Dense(1)(inputs)

model = keras.Model(inputs, outputs)

print(model)
```

```
<Functional name=functional, built=True>
```

Keras Layers:-

Layers are the fundamental building blocks of a neural network in Keras. Each layer performs a specific operation on the input data before passing it to the next layer.

Keras provides many built-in layers for creating different types of Deep Learning models. Some of the most commonly used layers are **Dense**, **Dropout**, and **Flatten**.

1. Dense():-

The `Dense()` layer is a fully connected layer where every neuron is connected to every neuron in the previous layer.

It is one of the most commonly used layers in Artificial Neural Networks (ANNs).

Syntax:

```
keras.layers.Dense(units, activation)
```

Example:

```
keras.layers.Dense(64, activation="relu")
```

```
from tensorflow import keras

layer = keras.layers.Dense(64, activation="relu")

print(layer)
```

```
<Dense name=dense_1, built=False>
```

2. Dropout():-

The `Dropout()` layer randomly disables a fraction of neurons during training. This helps reduce **overfitting** and improves the model's ability to generalize.

Syntax:

```
keras.layers.Dropout(rate)
```

Example:

```
keras.layers.Dropout(0.5)
```

```
from tensorflow import keras

layer = keras.layers.Dropout(0.5)

print(layer)
```

```
<Dropout name=dropout, built=True>
```

3. Flatten():-

The `Flatten()` layer converts multi-dimensional input data into a one-dimensional vector. It is commonly used before connecting convolutional layers to dense layers.

Syntax:

```
keras.layers.Flatten()
```

Example:

```
keras.layers.Flatten()
```

```
from tensorflow import keras

layer = keras.layers.Flatten()
```

```
print(layer)
```

```
<Flatten name=flatten, built=False>
```

Model Compilation:-

Before training a Keras model, it must be **compiled**. Model compilation configures how the model will learn from the training data.

The `compile()` method defines:

- **Optimizer:** Updates the model's weights.
- **Loss Function:** Measures the prediction error.
- **Metrics:** Evaluates the model's performance.

1. compile():-

The `compile()` method prepares the model for training by specifying the optimizer, loss function, and evaluation metrics.

```
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(2,))
])

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["accuracy"]
)

print("Model Compiled")
```

```
Model Compiled
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

2. Optimizer:-

An **Optimizer** updates the model's weights during training to minimize the loss function.

Some commonly used optimizers are:

- Adam
- SGD
- RMSprop

```
from tensorflow import keras

optimizer = keras.optimizers.Adam()

print(optimizer)
```

```
<keras.src.optimizers.adam.Adam object at 0x7fbbfaac8ad0>
```

3. Metrics:-

Metrics are used to evaluate the performance of a model during training and testing.

Some commonly used metrics are:

- Accuracy
- Precision
- Recall

```
from tensorflow import keras

metric = keras.metrics.Accuracy()

print(metric)
```

```
<Accuracy name=accuracy>
```

Model Training:-

After compiling the model, the next step is **training**. During training, the model learns patterns from the training data by adjusting its weights to reduce the loss.

Keras uses the `fit()` method to train a model.

Training Process

- The model receives input data.
- It makes predictions.
- The loss is calculated.
- The optimizer updates the weights.
- The process repeats for multiple epochs.

1. fit():-

The `fit()` method trains the model using the provided training data.

Syntax:

```
model.fit(x, y, epochs=10)
```

Parameters:

- **x:** Training input data.
- **y:** Target values.
- **epochs:** Number of complete passes through the dataset.
- **batch_size:** Number of samples processed before updating weights.

```
from tensorflow import keras
import numpy as np

x = np.random.rand(10, 2)
y = np.random.rand(10, 1)

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(2,))
])

model.compile(optimizer="adam", loss="mse")

model.fit(x, y, epochs=3)
```

```
Epoch 1/3
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 924ms/step - loss: 0.1996
Epoch 2/3
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - loss: 0.1984
Epoch 3/3
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.1972
<keras.src.callbacks.history.History at 0x7fbbfc7765a0>
```

2. Epochs:-

An **Epoch** is one complete pass of the entire training dataset through the neural network.

More epochs allow the model to learn better, but using too many may lead to **overfitting**.

Example:

```
epochs=5
```

```
from tensorflow import keras
import numpy as np

x = np.random.rand(10, 2)
y = np.random.rand(10, 1)

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(2,))
])

model.compile(optimizer="adam", loss="mse")

model.fit(x, y, epochs=5)
```

```
Epoch 1/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 525ms/step - loss: 0.1349
Epoch 2/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 51ms/step - loss: 0.1346
Epoch 3/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.1344
Epoch 4/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 0.1341
Epoch 5/5
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.1338
<keras.src.callbacks.history.History at 0x7fbbf86367e0>
```

3. Batch Size:-

Batch Size is the number of training samples processed before updating the model's weights.

Smaller batch sizes require less memory, while larger batch sizes may train faster.

Example:

```
batch_size=4
```

```
from tensorflow import keras
import numpy as np

x = np.random.rand(12, 2)
y = np.random.rand(12, 1)

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(2,))
])

model.compile(optimizer="adam", loss="mse")

model.fit(x, y, epochs=2, batch_size=4)
```

```
Epoch 1/2
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 1.0438
Epoch 2/2
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 1.0329
<keras.src.callbacks.history.History at 0x7fbbf8637620>
```

Model Evaluation:-

After training a model, it is important to evaluate its performance on unseen data. Model evaluation helps determine how well the model has learned from the training data.

Keras provides the `evaluate()` method to measure the model's performance using test data.

1. evaluate():-

The `evaluate()` method calculates the loss and performance metrics of a trained model using test data.

Syntax:

```
model.evaluate(x_test, y_test)
```

Returns:

- Loss
- Evaluation Metrics (such as Accuracy)

```
from tensorflow import keras
import numpy as np

x = np.random.rand(10, 2)
y = np.random.rand(10, 1)

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(2,))
])

model.compile(optimizer="adam", loss="mse")

model.fit(x, y, epochs=2, verbose=0)

model.evaluate(x, y)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 144ms/step - loss: 1.1229
1.1229256391525269
```

2. Loss:-

The **Loss** value indicates how far the model's predictions are from the actual values.

- Lower loss indicates better performance.
- The objective during training is to minimize the loss.

Example Output:

```
Loss: 0.25
```

```
from tensorflow import keras

loss = keras.losses.MeanSquaredError()

print(loss)
```

```
<LossFunctionWrapper(<function mean_squared_error at 0x7fbc2d1d7ce0>, kwargs={})>
```

Making Predictions:-

After training and evaluating a model, the final step is to use it for making predictions on new, unseen data.

Keras provides the `predict()` method to generate predictions. These predictions can be used in real-world applications such as image classification, spam detection, sentiment analysis, and price prediction.

1. predict():-

The `predict()` method uses a trained model to predict outputs for new input data.

Syntax:

```
model.predict(data)
```

Returns:

- Predicted values
- Probabilities (for classification models)

```
from tensorflow import keras
import numpy as np

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(2,))
])

data = np.array([[5, 6]])

print(model.predict(data))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 172ms/step
[[6.2997513]]
```

2. Predict Single Sample:-

A single data sample can also be passed to the `predict()` method. The input should be in two-dimensional form.

Example:

```
sample = [[2, 3]]
model.predict(sample)
```

```
from tensorflow import keras
import numpy as np

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(2,))
])

sample = np.array([[2, 3]])

print(model.predict(sample))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
[[-0.72294474]]
```

Saving & Loading Models:-

After training a model, it is often saved so that it can be reused later without retraining. Keras provides simple methods to save a trained model and load it whenever needed.

This is useful for deployment, sharing models, and continuing training.

1. save():-

The save() method stores the complete model, including its architecture, weights, and training configuration.

Syntax:

model.save("model.keras")

```
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(2,))
])

model.save("model.keras")
```

2. load_model():-

The load_model() function loads a previously saved Keras model.

Syntax:

keras.models.load_model("model.keras")

```
from tensorflow import keras

model = keras.models.load_model("model.keras")

print(model)
```

```
<Sequential name=sequential_8, built=True>
```

Callbacks:-

Callbacks are functions that are automatically executed during different stages of model training. They help monitor training, stop training early, save the best model, and perform other useful tasks.

1. EarlyStopping:-

EarlyStopping stops training when the model's performance stops improving. This helps prevent **overfitting** and saves training time.

Example:

keras.callbacks.EarlyStopping(patience=3)

```
from tensorflow import keras

callback = keras.callbacks.EarlyStopping(patience=3)

print(callback)
```

```
<keras.src.callbacks.early_stopping.EarlyStopping object at 0x7fbbf842da60>
```

2. ModelCheckpoint:-

ModelCheckpoint automatically saves the best model during training based on a selected performance metric.

Example:

keras.callbacks.ModelCheckpoint("best_model.keras")

```
from tensorflow import keras

callback = keras.callbacks.ModelCheckpoint("best_model.keras")

print(callback)
```

```
<keras.src.callbacks.model_checkpoint.ModelCheckpoint object at 0x7fbbf842e2d0>
```

Using Pretrained Models:-

Pretrained models are Deep Learning models that have already been trained on large datasets such as ImageNet. Instead of training a model from scratch, you can use these models for your own applications.

Keras provides many pretrained models through the **keras.applications** module. These models save time, require less training data, and often achieve high accuracy.

1. MobileNet:-

MobileNet is a lightweight Convolutional Neural Network (CNN) designed for mobile and embedded devices. It provides good accuracy while requiring fewer computational resources.

Example:

keras.applications.MobileNet()

```
from tensorflow import keras

model = keras.applications.MobileNet()
```

```
print(model)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet/mobilenet_1_0_224_tf.h5
17225924/17225924 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
<Functional name=mobilenet_1.00_224, built=True>
```

2. ResNet:-

ResNet (Residual Network) is a powerful Deep Learning model designed for image recognition tasks. It uses residual connections to enable the training of very deep neural networks.

Example:

keras.applications.ResNet50()

```
from tensorflow import keras

model = keras.applications.ResNet50()

print(model)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_ke
102967424/102967424 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
<Functional name=resnet50, built=True>
```

Conclusion:-

Keras is a simple, powerful, and beginner-friendly Deep Learning library that makes it easy to build, train, evaluate, and deploy neural networks. As the official high-level API of TensorFlow, it provides a clean interface with many built-in layers, optimizers, loss functions, callbacks, and pretrained models.